大模型推理的访存需求极其复杂
仿真器如何对 HBM / HBF / SRAM 精细建模

从每一次访存请求出发：拆成 burst / 页 / 端口，记住行与页的状态，
在共享的物理资源上排队计时，最后汇总成 Token 延迟与吞吐。

HBM
32 B burst · bank 行状态 · 命令时序

HBF
4 KiB 页 · plane 并行 · 页缓冲

SRAM
64 B burst · 端口流水线

访存耗时取决于“怎么访问”，不是一个带宽数能折算的

读 1 个 token 的 KV（256 B）

27.6×

HBM 37.8 ns | HBF 1,041 ns

HBF 必须把整页 4 KiB 读出，1 µs 的阵列读页时间被这 256 B 独自承担。

同样读 1 MiB：相邻存放 vs 分散存放

2.5×

相邻 2.93 TB/s | 分散 1.17 TB/s

分散后出现 12,288 次行冲突：bank 要先关掉旧行，再打开新行。

写 1 个 token 的 KV（256 B）

约 4,500×

HBM 22.4 ns | HBF 101.1 µs

不满一页的写要先读出旧页、合并后整页编程，编程一次 100 µs。

决定一次访存耗时的因素

单次访问多大	数据存放在哪	同时在途多少	读还是写	介质当前状态（开着哪一行、缓存着哪一页）

→ 仿真器逐请求、按介质的物理层级计算完成时间，而不是“字节数 ÷ 峰值带宽”。

探针：论文 Table I 的 HBM4（8 堆，12.8 TB/s）与 HBF（7 堆，读 11.2 TB/s）配置，冷启动单请求；数据 probe_results.json

同一个 Decode 步里，几类截然不同的访存同时发生、争用同一组资源

数据	单次访问有多大（每卡、每层）	访问特点	对存储的考验
权重	约 0.8 GB，大块连续	每生成一个 token 完整读一遍；只读	持续带宽；已开的行能否连续命中
KV 读	上下文长度 × 256 B（K、V 各一份） SHORT 1,660 token → 425 KB LONG 103.5K token → 26.5 MB	每个请求、每一层各读一段；长度随请求不同	短读：HBF 读页时间摊不薄 分散存放：HBM 行冲突
KV 写	每 token 每层 256 B（K、V 各一次）	极小、追加式、每一步都有	HBF 不满一页的写；读写方向切换
激活 / 中间结果	KB ~ MB 级	读写交替，生命周期短	容量放不放得下；端口争用
PD 导入的 KV	按 1 MiB 分段的大块写	与 Decode 同时进行	写带宽；与读争用同一批资源

关键：同一个数据，单次访问可以从 256 B 到 GB 级；同一时刻还有成百上千个请求在途。存储表现取决于这些访问的组合，而不是其中任何一种。

尺寸按 Llama 3.1 405B、BF16、TP8（每卡 1 个 KV 头 × 128 维）计算；SHORT / LONG 为论文负载 1,660 / 103,500 输入 token

每一次访存都走完同一条物理链路，时间由资源排队算出来

1 **算子**

Attention、GEMM 等声明要读写哪些缓冲区

→

2 **物理地址**

分配器给出地址：权重、每请求每层的 K/V、激活

→

3 **访存请求**

读/写 · 起始地址 · 字节数 · 到达时间

→

4 **介质核心**

拆成最小单位 → 映射到物理单元 → 查当前状态

→

5 **资源排队**

命令总线、数据线、plane、内部通道、链路、端口按时间表占用

→

6 **回写结果**

完成时间回到事件内核 → 算子结束 → TPOT / TTFT / 吞吐

④⑤ 是本次汇报的重点：HBM、HBF、SRAM 在这两步上建模的层级不同

常见简化

时间 = 字节数 ÷ 峰值带宽（+ 固定延迟）

与地址、单次大小、介质状态、同时在途数都无关

本仿真器

时间 = 沿依赖链、在共享资源时间表上排队得到的完成时刻

同样的字节数，因访问形态不同可以差几个数量级

代码：planner.py（算子与缓冲区）· memory_allocator.py · memory_types.AccessRequest · memory_mapping.py · dram_core.py / nand_core.py / sram_core.py · event_kernel.py

建模层级：HBM 到 32 B burst，HBF 到 4 KiB 页，SRAM 到端口

HBM	论文 HBM4 配置 · 8 堆
堆 Stack	× 8
通道 Channel	× 32 / 堆
伪通道 PC	× 2 / 通道，共 512 个
Bank Group	× 8 / PC
Bank	× 8 / 组
行 Row	1 KiB × 16,384 / bank
Burst（最小搬运单位）	32 B

状态	每个 bank 开着哪一行
排队	命令总线（2 个 PC 共用）、PC 数据线
时序	tRCD、tRP、RL、tRRD/tFAW、tCCD、刷新

HBF	论文 HBF 配置 · 7 堆
堆 Stack	× 7
Die	× 16 / 堆，共 112 个
Plane（独立阵列）	× 25 / die（共 2,800 个）
Block	256 页
页 Page（最小读出单位）	4 KiB

状态	每 plane 2 个页缓冲（LRU）
排队	plane、die 内部通道、主机链路
时序	读页 1 µs、编程 100 µs、擦除 2 ms

SRAM	论文 HBF+ 配置 · 8 × 40 MB
存储池	320 MB
端口	× 8，按 64 B 轮流条带
Burst（最小搬运单位）	64 B

状态	无行 / 页状态
排队	8 个端口（共 80 TB/s）、在途额度
时序	1 ns 流水线时延

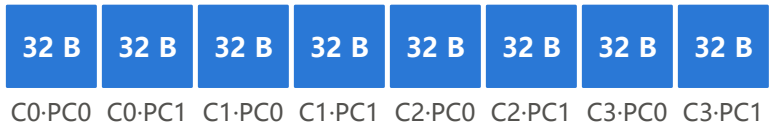
HBM 组织来自 JEDEC HBM4（JESD270-4A）；HBF 堆/die/plane 数与时延来自论文 Table I；SRAM 为论文 HBF+ 配置的研究参考；代码 hbm_geometry.py · memory_mapping.py

同一段 KV，在 HBM 里被切成 32 B 分散并行，在 HBF 里被整页读出

一个请求、一层的 K 缓冲区：按 token 顺序连续存放，每 token 256 B



HBM

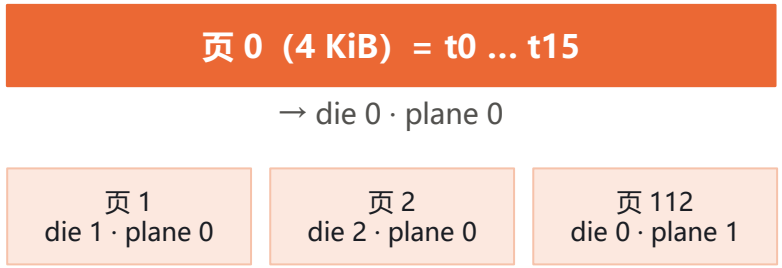


t0 的 256 B = 8 个 32 B burst

按 32 B 交织落到 4 个通道 × 2 个 PC，8 条数据通路同时传输。

冷启动读 t0：37.8 ns（8 个 bank 各开一次行）

HBF



t0 ~ t15 共用一个 4 KiB 页

读 t0 也要整页读出：1,041 ns；下一步读 t1 时，该页还在 plane 页缓冲里：0.37 ns。

相邻页依次落到不同 die，再换 plane

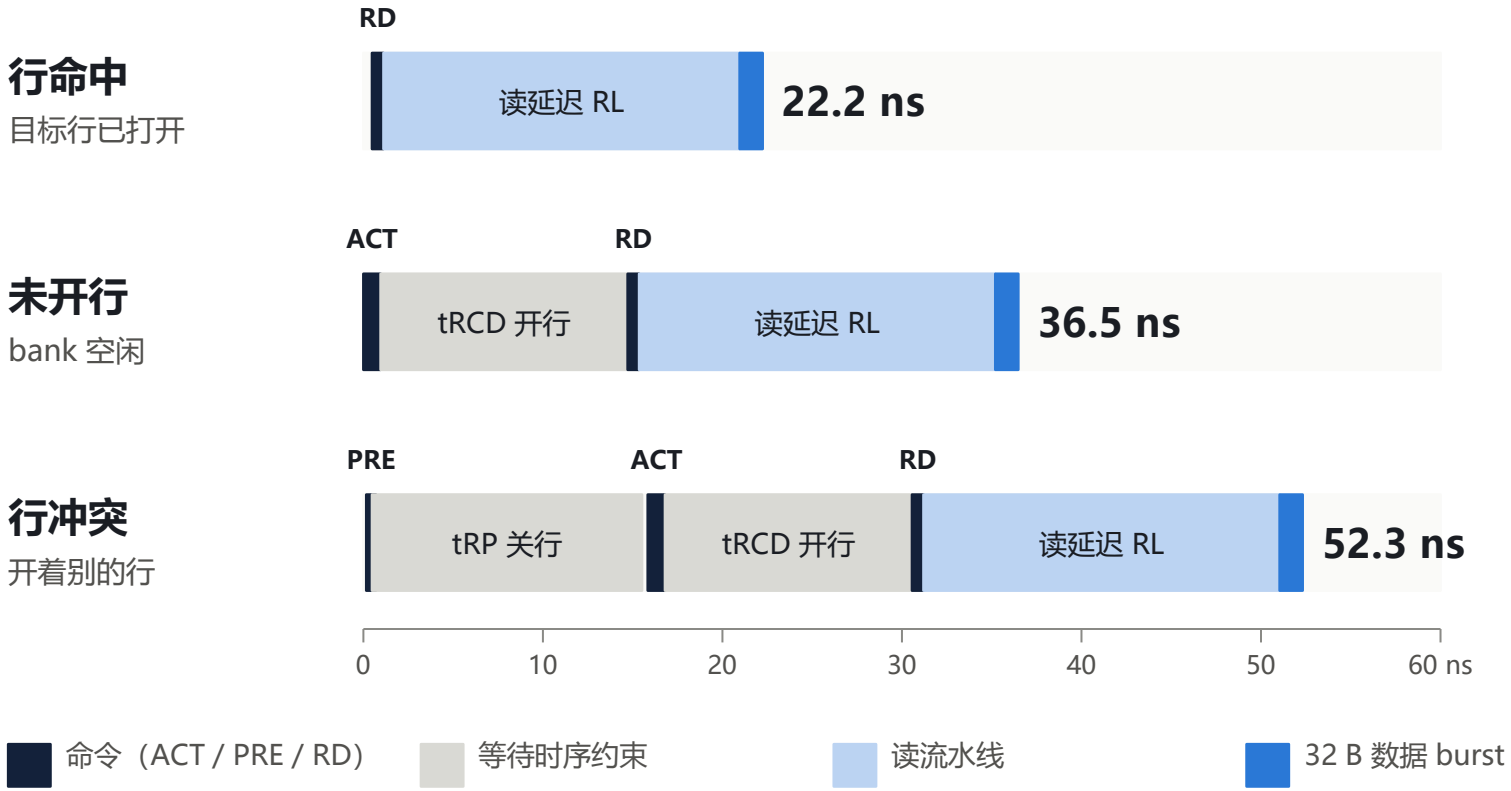
SRAM



t0 的 256 B = 4 个 64 B burst，轮流分到 4 个端口

无行/页状态：1.0 ns（1 ns 流水线 + 传输）

HBM：同样读 32 B，耗时取决于目标行是否已经打开



仿真器逐个 bank 记住“开着哪一行”：上一个访问留下的行状态，决定下一个访问走哪条路径。

此外还要同时满足

开行速率

同一 PC 相邻 ACT ≥ 4 / 6 ns；任意 20 ns 内最多 4 次 ACT

列命令间隔

同 bank group 2.56 ns，跨组 1.28 ns；地址映射让连续 burst 轮换 bank group

共享命令总线

同一物理通道的 2 个 PC 共用行 / 列命令总线

刷新

每 3.9 µs 全 bank 刷新一次，期间该 PC 不可访问

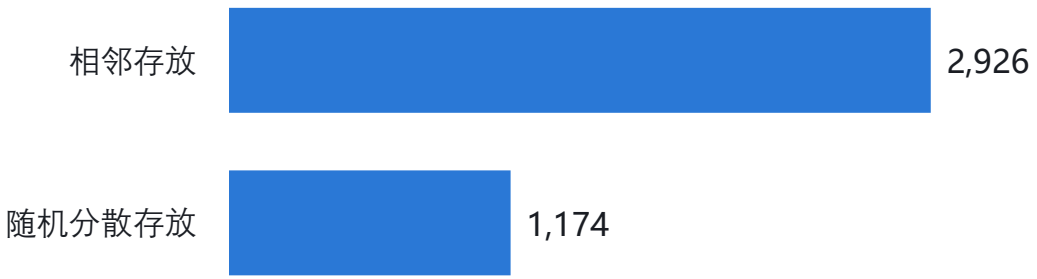
其他

读写换向、写恢复、最长开行时间

仿真器实测（DramCore，jedec_hbm 模式）：HBM4 研究参考时序 tCK 0.64 ns、tRCD 14 ns、tRP 15.36 ns、读延迟 20.48 ns；tRRD/tFAW 借用 HBM3E 参考包，非厂商标定

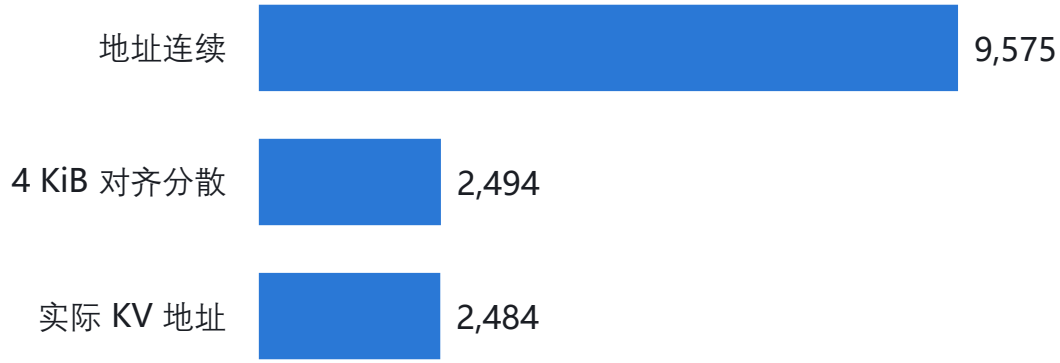
HBM：数据放在哪里、一次读多大，直接决定带宽

同样读 1 MiB（256 次 × 4 KiB），有效带宽 GB/s



相邻：32,768 次开行，0 次冲突
分散：20,480 次开行 + 12,288 次行冲突（先关再开）

405B 真实 KV 读（B=64，第 63 层，128 条 × 470 KB），GB/s



同样的字节数，真实 KV 地址只有连续存放的 26%

一个 32 B burst 只用到 8 B

搬运字节不变，有效带宽 47.2 → 11.8 GB/s，只剩 1/4。仿真器按 burst 计搬运量，不按请求字节数。

单次读越大，已开的行被复用越多

1 MiB 冷读每个 bank 只用一次行：2.9 TB/s；26.5 MB 读 10.7 TB/s，96% 的 burst 命中已开的行。

左：仿真器探针（冷启动、1 MiB）；右：docs/exploring_hbf_native_comparison.md 记录的 405B 真实地址实测（2026-10-22，提交 add4a5d）

HBF：一次读要占用三段资源，最小单位是 4 KiB 页



读放大

只要 256 B，也要读出并搬运整页 4 KiB

16×

单个 token 的读：搬运 4,096 B，有用 256 B

并行靠“铺页”

连续的页依次落到 112 个 die，再换 plane

1.13 µs

1 MiB 连续读：256 页同时在不同 plane 上读

位置也有影响

页落到同一个 plane，读页只能排队

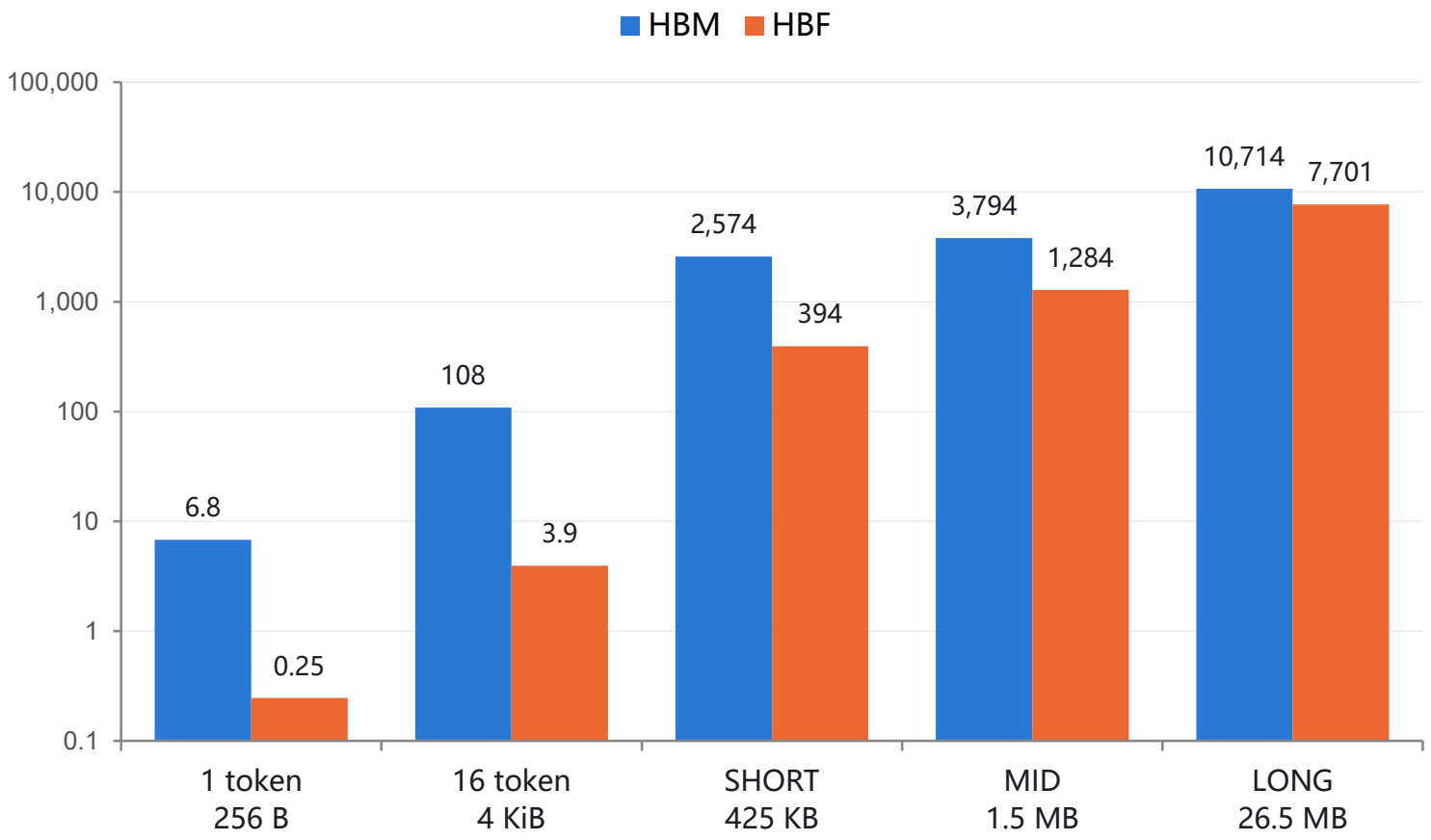
3.25 µs

256 个随机页：有的 plane 分到 3 页，只能串行读 3 次

仿真器 NandCore（generic_nand）：页读 1 µs；内部通道每 die 100 GB/s；主机读 11.2 TB/s；本配置主机传输粒度 4 KiB（OCP HBF 规范允许 64 B，可配置）

HBF：KV 读得越大、越并发，越接近 HBM；短读时劣势最大

单次读的有效带宽（GB/s，对数刻度）



HBF 耗时是 HBM 的几倍

1 token（256 B）	**27.6×**
16 token（4 KiB）	**27.6×**
SHORT（425 KB）	**6.53×**
MID（1.5 MB）	**2.95×**
LONG（26.5 MB）	**1.39×**

批量也能把读“凑大”

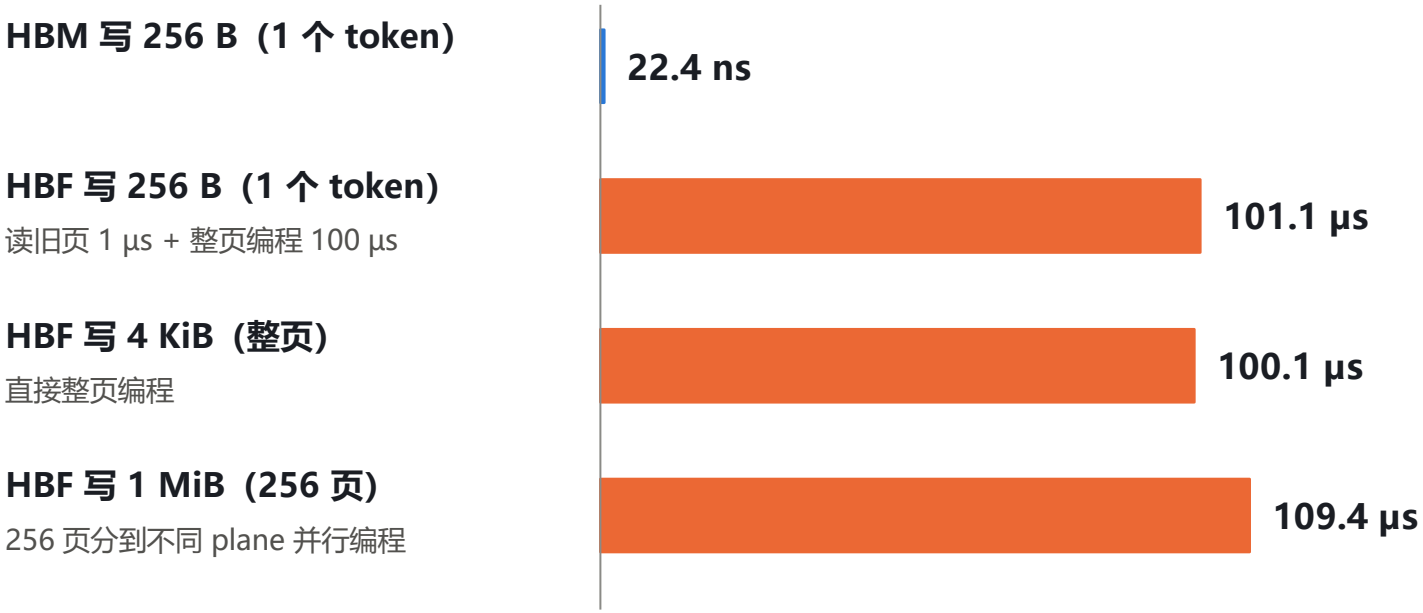
64 个 SHORT 请求同时读：HBF 7.75 TB/s，HBM 10.73 TB/s，只差 1.38×。

劣势的本质是“同一时刻在途的页太少，1 µs 读页时间摊不薄”。

仿真器探针：单个冷请求，读一个请求一层的 K（405B、TP8 每 token 256 B）；批量行为 64 个 SHORT 请求同时读各自的 K 缓冲区

HBF：每 token 的小写入代价最大，“写合并”作为显式策略建模

完成时间（线性刻度，0 ~ 110 µs）



Decode 每生成 1 个 token，每层要给 K、V 各追加 256 B。如果直接写入 HBF，每次都要付一次约 100 µs 的整页编程，比 HBM 慢约 4,500 倍。

仿真器提供的写合并策略

1. **不满一页先暂存**
   KV 追加先写进主机 DRAM，经真实链路传输并占用暂存容量
2. **写满再整页写入**
   凑满 4 KiB 后读出暂存页，整页编程到 HBF
3. **读时两处合并**
   Attention 同时读 HBF 中已写入的部分和主机 DRAM 里的尾页
4. **成本全部计入**
   暂存容量、链路、读改写、空间回收与擦除都按物理操作计费

仿真器探针（单个冷请求）：页编程 100 µs、读改写策略；主机写暂存见 docs/hbf_host_staging.md（严格 OCP HBF 预设，可选）

SRAM：物理上没有行和页，模型只保留真正起作用的容量与端口

模型保留的要素

容量

320 MB，存放中间数据（激活、工作区）；放不下就要退回别的介质

端口

8 个端口，地址按 64 B 轮流条带；每端口 10 TB/s，读写共用

流水线时延

1 ns，先于数据传输；不按字节串行累加

在途额度

最多 256 个请求同时在途，超出就排队

不建的部分

无行/页状态，不因地址位置产生额外惩罚

读 1 个 token（256 B）

1.0 ns

基本只有 1 ns 流水线时延

读 26.5 MB（LONG 上下文的 K）

79.8 TB/s

已接近 80 TB/s 峰值，大读不受位置影响

32,768 次分散的 32 B 读

搬运 2×

每次至少搬一个 64 B burst；小请求还受 256 在途额度限制

仿真器 SramCore；参数为论文 HBF+ 配置的研究参考（8 堆 × 40 MB，总 80 TB/s，1 ns 流水线，256 在途额度）；GPU 片上 L1/L2 属于算子模型，不在本页范围

按峰值带宽折算，小读写与分散访问会被低估 10 倍到数万倍

场景	按峰值折算	仿真器结果	相差
SRAM 读 26.5 MB	331 ns	332 ns	**1.0×**
HBM 连续读 26.5 MB	2,070 ns	2,473 ns	**1.2×**
HBF 连续读 26.5 MB	2,366 ns	3,441 ns	**1.5×**
HBM 读 1 MiB（256 × 4 KiB 分散）	82 ns	893 ns	**10.9×**
HBF 读 1 个 token（256 B）	0.02 ns	1,041 ns	**约 4.6 万×**
HBF 写 1 个 token（256 B）	2 ns	101.1 µs	**约 4.4 万×**

细节会改变端到端结论

405B、HBM4、SHORT、B=512 的 Decode：仅修正 HBM 地址映射（连续 burst 轮换 bank group）和行命令调度（提前关行），每层由 1.275 ms 降到 0.437 ms，TPOT 约 160 ms → 55 ms。同一份负载，层级细节建错，结论可以差 3 倍。

“按峰值折算” = 字节数 ÷ 标称峰值（HBM 12.8 TB/s；HBF 读 11.2 TB/s、写 112 GB/s；SRAM 80 TB/s）； “仿真器” = 同一场景的探针实测

当前建模深度与边界：机制已经实现，参数尚未经器件标定

介质	已经建模	参数来源	未建模 / 使用时注意
HBM	地址映射到堆 / 通道 / PC / bank / 行 / 列；ACT、PRE、RD、WR、全 bank 刷新命令；每 bank 行状态；共享命令总线与数据线；tRRD / tFAW / tCCD / 换向	组织来自 JEDEC HBM4 协议；时序为参考工程值与研究假设	厂商地址映射与速度档、PHY 训练、按 bank 刷新；调度按请求而非逐命令重排
HBF	页级读 / 编程 / 擦除；plane 并行、die 内部通道、主机链路；每 plane 2 页缓冲；不满页读改写；在途额度；可选主机写暂存	堆 / die / plane 数与读写时延来自论文 Table I；块几何、擦除、内部带宽为研究假设	论文配置（generic NAND）无 FTL / 垃圾回收；ECC、数据保持未建；磨损为可选简化模型
SRAM	容量、8 端口条带、64 B burst、1 ns 流水线、在途额度	容量来自论文；端口数、时延为研究假设	按池化处理，未区分各堆私有空间与路由

结果应这样读：

同一套物理机制下比较不同介质与部署是可靠的；绝对数值取决于未标定参数。拿到目标器件的时序表后，替换参数即可，不需要改模型结构。

依据：hbm_geometry.py / docs/hbm_geometry.md · nand_core.py 类说明 · sram_core.py · scripts/exploring_hbf_paper_config.py；所有配置 vendor_calibrated = False